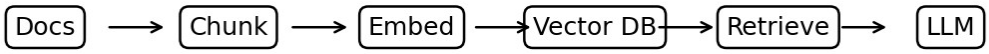
32. VECTOR DATABASES

Embeddings • Similarity Search • ANN • FAISS • Chroma • Pinecone • RAG

Vector database RAG içinde nerede?



metadata filter + hybrid search + reranking

Bu modülün amacı
Vector database kavramını yalnız ürün isimleri üzerinden değil; embedding uzayı, similarity search, approximate nearest neighbor index, metadata filter, hybrid retrieval, reranking ve RAG production tasarımı üzerinden öğrenmek.

Kazanım	Modül sonunda
Embedding	Vector representation ve semantic similarity mantığını açıklayabilirsin.
Similarity	Cosine, dot product ve L2 farklarını doğru yorumlayabilirsin.
Index	Exact search ile ANN/HNSW/IVF yaklaşımını ayırabilirsin.
Vector DB	FAISS, Chroma ve managed servis mantıklarını karşılaştırabilirsin.
Retrieval	Metadata filter, hybrid search ve reranking tasarlayabilirsin.
RAG	Ingestion-index-query-evaluation yaşam döngüsünü kurabilirsin.

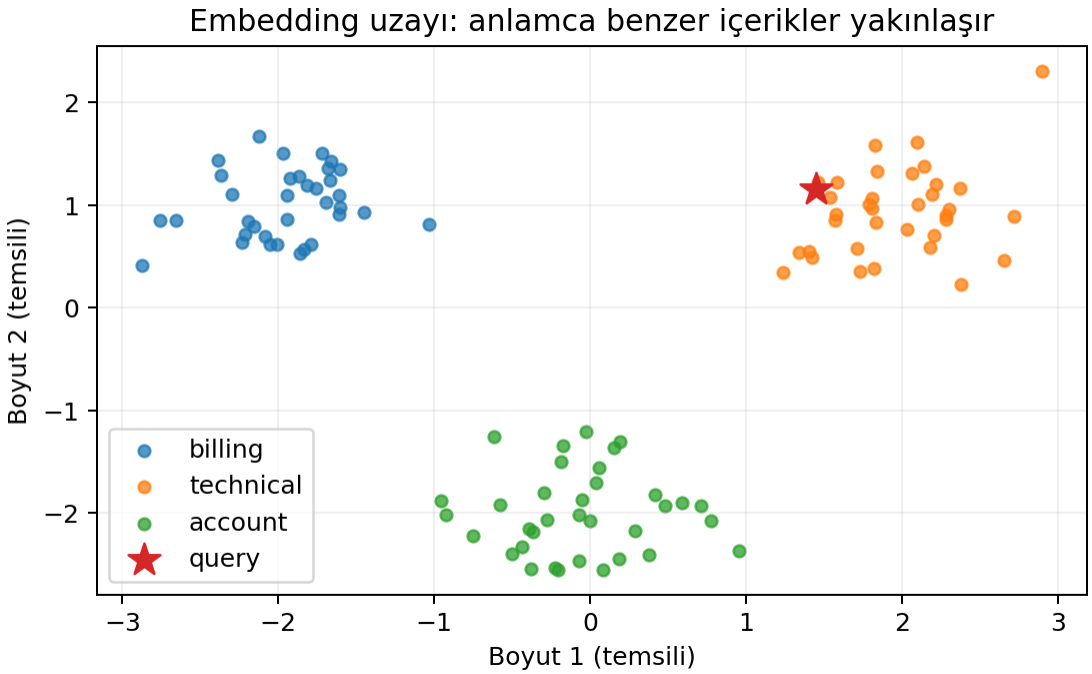
1. Vector database nedir?

Vector database, yüksek boyutlu embedding vektörlerini saklamak, indexlemek ve query embeddingine en yakın vektörleri düşük latency ile bulmak için tasarlanmış sistemdir.

Klasik DB sorgusu	Vector DB sorgusu
WHERE category = "refund"	Bu soruya anlamca en yakın chunklar hangileri?
Exact field equality	Semantic similarity
B-tree/hash index	ANN/vector index
Rows/documents	Vectors + metadata + payload

Önemli
Vector database relational database yerine geçmez. Semantic retrieval gereken katmana eklenen özel bir veri erişim sistemidir.

2. Embedding nedir?



Embedding, metin/görsel/ürün/kullanıcı gibi bir nesneyi dense numeric vector ile temsil eder. Öğrenilmiş embedding uzayında benzer anlam veya davranış taşıyan nesnelerin yakın olması hedeflenir.

Boyut
Embedding dimension model tarafından belirlenir. Daha yüksek dimension otomatik olarak daha kaliteli retrieval anlamına gelmez.

3. Query embedding ve document embedding

Tür	Kaynak	Amaç

Document embedding	Chunk/document	Indexte saklanır
Query embedding	Kullanıcı sorusu	Nearest-neighbor search
Item embedding	Ürün/içerik	Recommendation/search
User embedding	Kullanıcı davranışı	Personalized retrieval

Aynı uzay
Query ve document embeddingleri karşılaştırılacaksa modelin bu iki input tipini uyumlu uzaya taşıması gerekir.

4. Similarity: cosine, dot product, L2

Metric	Ne ölçer?	Not
Cosine similarity	Vektörler arası açı	Magnitude etkisini azaltır
Dot product	Yön + magnitude	Normalize edilmezse magnitude etkili
Euclidean / L2	Uzaysal mesafe	Küçük değer daha yakın

```
cosine(a,b) = (a · b) / (||a|| ||b||)
```

Model dokümanını kontrol et
Embedding modelinin önerdiği similarity metric tercih edilmelidir. Normalize edilmiş vectorlarda cosine ve dot product rankingleri eşdeğer olabilir.

5. Vector normalization

```
import numpy as np

v = np.array([1.2, 3.4, 0.7])
v = v / np.linalg.norm(v)
```

Neden?
Cosine araması yapmak istiyorsan unit-normalized vectorlar dot-product indexleriyle verimli kullanılabilir.

6. Exact nearest-neighbor search

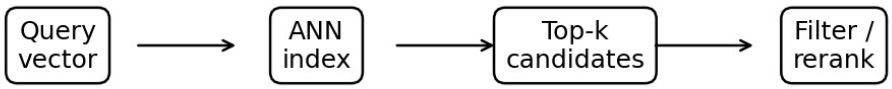
Query vector ile tüm N vector arasındaki similarity hesaplanır. Küçük koleksiyonlarda basit ve maksimum recall sağlar; ölçek büyüdükçe compute maliyeti artar.

```
scores = embeddings @ query_vector
top_idx = np.argsort(scores)[-5:][::-1]
```

Complexity sezgisi
Milyonlarca vector ve yüzlerce dimension olduğunda her sorguda tüm vectorları taramak pahalı hale gelir.

7. Approximate Nearest Neighbor (ANN)

Vector search akışı



Exact search yerine approximate nearest neighbor ile hız/recall dengesi

Trade-off
ANN exact sonucu garanti etmek yerine çok daha düşük latency için küçük recall kaybını kabul eder.

8. Recall@k - latency trade-off

Ayar	Recall	Latency	Memory
Daha geniş search	Artar	Artar	Benzer
Daha agresif approximation	Azalabilir	Düşer	Değişir
Daha büyük graph/index	Genelde artar	Düşebilir	Artar

Evaluation
Vector DB tuning yalnız latency ile yapılmaz. Retrieval recall ve downstream answer quality birlikte izlenmelidir.

9. HNSW



Üst seviyeler hızlı navigasyon, alt seviye hassas komşuluk araması sağlar.

Hierarchical Navigable Small World, çok katmanlı graph üzerinden yakın komşulara hızlı ulaşmayı hedefleyen yaygın ANN index ailesidir.

Parametre fikri	Etkisi
M	Node başına graph bağlantı yoğunluğu
efConstruction	Index build quality/compute
efSearch	Query sırasında search genişliği

Sezgi
efSearch artırılırsa genellikle recall artar ama latency de artar.

10. IVF / inverted file index

Vector uzayı coarse clusterlara ayrılır. Query önce en yakın clusterları bulur, sonra yalnız ilgili clusterlardaki candidate vectorlar taranır.

Aşama	İşlem
Train	Coarse centroids öğren
Assign	Vectorları listelere ata
Query	En yakın nprobe listeleri ara
Rank	Candidate vectorları similarity ile sırala

11. Product Quantization (PQ)

High-dimensional vectorları düşük-bit kodlarla yaklaşık temsil ederek memory kullanımını ve distance compute maliyetini azaltır.

Trade-off
Compression artarsa memory azalır fakat similarity hassasiyeti düşebilir. Billion-scale search sistemlerinde değerlidir.

12. Flat, HNSW, IVF/PQ karşılaştırması

Index	Artı	Eksi	Ne zaman?
Flat	Exact recall	Yavaş/ölçek maliyeti	Küçük dataset veya benchmark
HNSW	Yüksek recall + hızlı	Memory yüksek olabilir	Interactive retrieval
IVF	Büyük koleksiyonda verimli	Training/tuning gerekir	Large-scale search
IVF+PQ	Memory çok düşer	Approximation artar	Çok büyük corpus

13. Vector kaydı nasıl görünür?

```
{
  "id": "doc_42_chunk_7",
  "vector": [0.12, -0.08, ...],
  "text": "Refund policy ...",
  "metadata": {
    "department": "support",
    "language": "tr",
    "version": 3
  }
}
```

ID stratejisi
Stable document/chunk ID; re-index, delete ve deduplication süreçlerini kolaylaştırır.

14. Metadata filtering

Semantic similarity tek başına yeterli değildir. Language, tenant, product, date, access scope gibi metadata ile candidate uzayı sınırlandırılabilir.

```
filter = {
  "language": "tr",
  "department": "finance"
}
```

Security
Multi-tenant RAG sisteminde erişim yetkisi yalnız prompt ile değil retrieval/filter katmanında enforce edilmelidir.

15. Pre-filter vs post-filter

Yaklaşım	Artı	Risk
Pre-filter	Yetkisiz/noisy candidate baştan elenir	Index desteğine bağlı

Post-filter	Basit implementasyon	Top-k sonrası yeterli sonuç kalmayabilir

Production
Access-control filtreleri mümkün olduğunca retrieval öncesinde veya index-native şekilde uygulanmalıdır.

16. Chunking ve vector DB

Karar	Etkisi
Chunk çok küçük	Context parçalanır, index büyür
Chunk çok büyük	Semantic signal seyrelir
Overlap fazla	Duplicate retrieval artar
Heading-aware	Doküman yapısını korur

Chunking hyperparameterdır
Chunk size/overlap retrieval evaluation ile seçilmeli; sabit 500 token evrensel doğru değildir.

17. Ingestion pipeline

- Kaynağı oku ve temizle
- Document ID/version üret
- Chunklara ayır
- Embedding batch üret
- Metadata ekle
- Vector DB upsert
- Index health doğrula
- Evaluation queryleriyle smoke test yap

18. Upsert, update ve delete

Operasyon	Not
Upsert	ID varsa güncelle, yoksa ekle
Delete	Belge silindiğinde chunks da silinmeli
Versioning	Eski versionın stale retrieval yapmasını önle
Re-embed	Embedding model değişince migration gerekir

19. Embedding model versioning

Farklı embedding modellerinin vectorları aynı anlam uzayında olmayabilir. Model sürümü değişince query ve document vectorları uyumlu şekilde yeniden üretilmelidir.

Migration
Blue/green index yaklaşımı: yeni embedding modeliyle ikinci index kur, evaluate et, sonra trafik geçir.

20. FAISS mantığı

FAISS, vector similarity search ve clustering için optimize edilmiş bir kütüphanedir. Local/in-process index sağlar; metadata, auth, multi-tenant lifecycle gibi database özelliklerini kendin tasarlarsın.

```
import faiss
import numpy as np

d = 768
index = faiss.IndexFlatIP(d)

index.add(doc_vectors.astype("float32"))
D, I = index.search(query_vector[None].astype("float32"), k=5)
```

FAISS != managed DB
FAISS güçlü bir vector index kütüphanesidir; tam database/servis katmanı değildir.

21. Chroma mantığı

Chroma, özellikle local prototyping/RAG için collection, embedding, document ve metadata abstractionı sağlayan vector store yaklaşımıdır.

```
collection.add(
    ids=["c1"],
    documents=["refund policy"],
    metadatas=[{"lang":"en"}]
)

results = collection.query(
    query_texts=["money back"],
    n_results=5
)
```

Kullanım
Hızlı prototip ve küçük/orta local uygulamalarda rahat; production ihtiyaçları için persistence/concurrency/ops gereksinimlerini ayrıca değerlendir.

22. Managed vector database mantığı

Özellik	Managed servis avantajı
Scaling	Sharding/replication otomasyonu
Availability	Managed durability/HA
Metadata	Native filters
Operations	Metrics, backups, upgrades
Security	Network/auth controls

Pinecone benzeri yaklaşım
Managed servis seçimi operasyon kolaylığı sağlar; maliyet, vendor lock-in ve data residency trade-offları değerlendirilir.

23. FAISS vs Chroma vs managed service

Seçenek	En güçlü yön	Tipik kullanım
FAISS	Index kontrolü/performance	Custom backend, offline/local
Chroma	Developer ergonomisi	Prototype/local RAG
Managed vector DB	Operational simplicity	Production hosted retrieval

Doğru soru
Hangi ürün daha iyi? yerine collection size, QPS, latency SLA, filters, multi-tenancy, ops ekip kapasitesi ve maliyet nedir?

24. Hybrid search

Dense embedding retrieval ile lexical BM25/keyword retrieval birleştirilir.

Dense iyi	Lexical iyi
Paraphrase / semantic match	Exact product code
Synonym	Error ID / SKU
Concept similarity	Nadir proper noun

Sonuç
Gerçek enterprise search/RAG sistemlerinde hybrid retrieval sıkça dense-only yaklaşımı geçer.

25. Score fusion

Dense ve sparse retriever farklı score ölçekleri kullanabilir. Rank fusion ile sonuç sıraları birleştirilebilir.

```
# Reciprocal Rank Fusion fikri
score(doc) += 1 / (k + rank_dense)
score(doc) += 1 / (k + rank_sparse)
```

RRF
Score normalization gereksinimini azaltan sağlam rank fusion yöntemlerinden biridir.

26. Reranking

Fast retriever top-20/top-100 candidate getirir. Daha pahalı cross-encoder/reranker query-document çiftlerini birlikte okuyup top-5 gibi final liste üretir.

Katman	Amaç
ANN retriever	Yüksek recall, düşük latency
Reranker	Yüksek precision
LLM	Final generation

İki aşamalı search
Retrieval geniş tut, reranking ile hassaslaştır.

27. Maximal Marginal Relevance (MMR)

Sadece query similarity değil, seçilen chunks arasındaki redundancy de dikkate alınır. Böylece birbirinin kopyası 5 chunk yerine daha çeşitli context elde edilir.

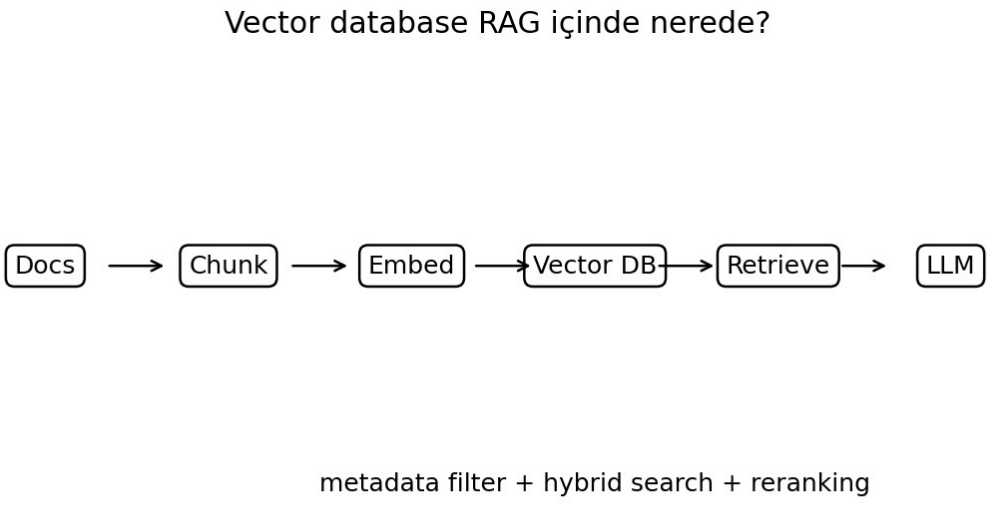
RAG context
MMR özellikle overlap yüksek corpuslarda duplicate context'i azaltabilir.

28. Query expansion / rewriting

- Short query -> daha açıklayıcı search query
- Acronym expansion
- Multi-query retrieval
- Language normalization

Risk
Query rewrite asıl kullanıcı niyetini değiştirebilir; original query de pipeline'da korunmalıdır.

29. RAG retrieval pipeline



- User query

- Query embedding / rewrite
- Metadata filter
- ANN top-k retrieval
- Hybrid fusion
- Rerank
- Context assembly
- LLM generation
- Citation mapping

30. Top-k seçimi

k çok düşük	k çok yüksek
Relevant bilgi kaçabilir	Noisy context artar
Recall düşebilir	Token maliyeti/latency artar
Generator bilgi bulamaz	Conflicting chunks gelebilir

Optimize et
Retriever top-k ile final prompta giren top-n ayrı hyperparameter olabilir.

31. Retrieval evaluation

Metric	Soru
Recall@k	Relevant chunk top-k içinde mi?
Precision@k	Top-k ne kadar temiz?
MRR	İlk relevant sonuç ne kadar yukarıda?
nDCG	Relevance derecelerini sıralama kalitesiyle birleştirir
Hit rate	En az bir doğru sonuç geldi mi?

Ground truth
Query -> relevant document/chunk mappinglerinden oluşan golden retrieval seti gerekir.

32. End-to-end RAG evaluation

Katman	Metric
Retrieval	Recall@k / MRR
Reranking	nDCG / precision@k
Answer	Correctness / task rubric
Grounding	Context-supported claims
Citation	Citation correctness
Operations	Latency / cost / failure rate

33. Common retrieval failures

Failure	Muhtemel neden
Relevant chunk hiç gelmiyor	Chunking/embedding/index/query sorunu

Yanlış tenant sonucu	Metadata/access filter bug
Aynı chunk tekrar tekrar	Overlap/dedup sorunu
Exact code bulunmuyor	Dense-only retrieval
Eski politika geliyor	Version/delete pipeline sorunu
Similarity iyi ama cevap kötü	Generation/context assembly sorunu

34. Deduplication

- Stable content hash
- Near-duplicate embedding similarity
- Canonical URL/document ID
- Version-aware indexing

Neden?
Duplicate chunks retrieval diversity'yi bozar ve context window'u gereksiz tüketir.

35. Multi-tenancy

Strateji	Trade-off
Tenant başına index/namespace	Güçlü izolasyon, operasyon artar
Shared index + tenant metadata	Kolay ölçek, filter correctness kritik
Hybrid	Büyük tenant ayrı, küçükler shared

Security requirement
Tenant filter kullanıcıdan gelen prompt stringine bırakılamaz; backend authorization context'inden üretilmelidir.

36. Sharding ve replication

Corpus tek node kapasitesini aşınca vectorlar shardlara dağıtılır. Replication availability ve read throughput artırabilir.

Operational design
Index type kadar shard count, replication, warmup ve rebalancing de p95 latency üzerinde etkilidir.

37. Memory hesabı

Raw float32 vector memory yaklaşık N x dimension x 4 byte.

```
# 10M vector x 768 dim x 4 byte
10_000_000 * 768 * 4 / 1024**3
# yaklaşık 28.6 GiB raw vector memory
```

Index overhead
HNSW graph, metadata ve replicas raw vector belleğine ek maliyet getirir. PQ gibi compression teknikleri burada önem kazanır.

38. Latency breakdown

Aşama	Latency kaynağı
Embedding API/model	Query encoding
Network	Client -> vector DB
ANN	Index traversal
Metadata	Filter processing
Reranker	Cross-encoder compute
LLM	Generation

Measure each stage
Sadece total latency ölçersen bottleneck'in embedding mi search mü reranker mı olduğunu bilemezsin.

39. Batch indexing

- Embeddingleri batch üret
- Rate limit/backoff kullan
- Upsert batch size tune et
- Checkpoint/progress sakla
- Partial failure retry et

Idempotency
Aynı batch tekrar işlendiğinde duplicate üretmemesi için deterministic IDs ve upsert kullan.

40. Online vs offline indexing

Tür	Örnek
Offline	Gecelik doküman rebuild
Near real-time	CMS update sonrası birkaç dakika
Streaming	Yeni event/item hemen searchable

41. Data freshness

KPI	Anlam
Index lag	Source update -> searchable olma süresi
Stale hit rate	Eski version retrieval oranı
Failed upsert rate	Indexing hataları
Delete lag	Silinen içeriğin indexte kalma süresi

RAG quality = data quality
Güncel olmayan vector index, güçlü LLM ile bile yanlış cevap üretir.

42. Observability

- QPS / p50 / p95 / p99 search latency
- No-result rate
- Filter cardinality
- Recall regression on golden set
- Index size / memory
- Embedding version distribution
- Top retrieved documents/chunks

43. Security ve privacy

Risk	Kontrol
Unauthorized retrieval	ACL/metadata filters
Sensitive embeddings	Encryption/access control
Deleted source still indexed	Delete propagation
Cross-tenant leak	Strong tenant isolation
Prompt injection in docs	Treat retrieved text as untrusted

Embedding anonim değildir
Embeddingden ham metni doğrudan okumak zor olsa da hassas bilgi taşıyan representationlar güvenli veri olarak yönetilmelidir.

44. Backup ve rebuild

Vector indexi source-of-truth olmak zorunda değildir. Raw documents + metadata + embedding config saklanırsa index yeniden üretilebilir.

Rebuildability
Production tasarımında indexi tamamen silip deterministik olarak yeniden kurabilmek güçlü bir operasyonel özelliktir.

45. Embedding drift

Embedding model veya source distribution değiştiğinde retrieval davranışı değişebilir.

- Golden queries ile periodic evaluation
- Embedding version tag
- A/B index test
- Query distribution monitoring

46. Vector DB ve recommendation

Vector search yalnız RAG değildir. Item-to-item similarity ve user-to-item retrieval recommendation sistemlerinde de kullanılır.

Query	Index
User embedding	Item embeddings
Current product	Similar product embeddings
Image embedding	Catalog image embeddings

47. Vector DB ve anomaly/dedup search

- Near-duplicate documents
- Similar incident tickets
- Image duplicate detection
- Known fraud pattern lookup

Genel pattern
Elinde query representation ve büyük candidate corpus varsa nearest-neighbor search düşün.

48. SQL/relational DB + vector extension

Bazı sistemler vector type/index özelliklerini relational database içine ekler. Böylece transaction/metadata joins ile vector search aynı platformda çalışabilir.

Ne zaman?
Collection orta ölçekliyse ve mevcut relational altyapı güçlü ise ayrı vector DB yerine vector-capable relational DB operasyonu sadeleştirebilir.

49. Vector DB seçme checklist

- Corpus size ve growth nedir?
- Embedding dimension nedir?
- QPS ve p95 latency hedefi nedir?
- Metadata filtering karmaşıklığı nedir?
- Hybrid search gerekir mi?
- Multi-tenancy ve ACL gereksinimi nedir?
- On-prem/cloud zorunluluğu var mı?
- Backup/rebuild strategy nedir?
- Ops ekibi index yönetebilir mi?
- Maliyet/QPS beklentisi nedir?

50. Common anti-patterns

- Similarity score yüksekse cevabı kesin doğru sanmak.
- Embedding model değiştirip eski vectorları kullanmak.
- Chunk ID/version yönetmemek.
- Test set olmadan index parametrelerini tuning etmek.
- Dense-only retrieval ile SKU/error code aramak.
- ACL filterı prompt ile uygulamak.
- Top-k değerini keyfi seçmek.
- Duplicate chunksı indexe doldurmak.
- Vector DB latencyyi LLM latency içinde kaybetmek.
- Source document silinince vectorları silmemek.

51. FAISS mini örneği

```
import faiss
import numpy as np

# normalized embeddings -> inner product ~ cosine
xb = doc_vectors.astype("float32")
faiss.normalize_L2(xb)

index = faiss.IndexFlatIP(xb.shape[1])
index.add(xb)

q = query_vector.astype("float32")[None]
faiss.normalize_L2(q)

scores, ids = index.search(q, 5)
```

Baseline
Önce exact/flat retrieval ile kalite baselinei kurmak, sonra ANN indexe geçip recall/latency trade-offunu ölçmek iyi pratiktir.

52. RAG retriever pseudo-code

```
def retrieve(query, tenant_id):
    q = embed(query)

    candidates = vector_db.search(
        vector=q,
        top_k=30,
        filter={"tenant_id": tenant_id}
    )

    candidates = hybrid_fuse(
        query, candidates
    )

    top = rerank(query, candidates)[:6]
    return top
```

Layered design
Embedding, retrieval, fusion, reranking ve context assembly ayrı component olursa evaluation ve debugging kolaylaşır.

53. Mülakat soruları

Soru	Kısa cevap
Vector database nedir?	Embedding vectorlarını ANN index ile semantic search için saklayan sistem.
Cosine vs dot product?	Cosine açıyı; dot yön+magnitude etkisini ölçer.
ANN neden?	Exact search maliyetini düşürmek için approximate nearest neighbor.
HNSW nedir?	Hierarchical graph tabanlı ANN index.
IVF nedir?	Coarse cluster/list üzerinden candidate alanını daraltır.
PQ nedir?	Vectorları quantize ederek memory/search maliyetini azaltır.
Metadata filter neden?	Scope, language, tenant ve access kısıtları.
Hybrid search?	Dense vector + lexical BM25/keyword.
Reranker?	Initial candidatesı daha pahalı modelle yeniden sıralar.
Top-k nasıl seçilir?	Golden set recall + context cost/latency ile tune edilir.
FAISS database mi?	Esasen vector search/index kütüphanesidir.
Embedding model değişirse?	Corpus yeniden embed/index edilmelidir.
Recall@k ne ölçer?	Relevant itemların top-k içinde bulunma oranı.
Multi-tenant risk?	Cross-tenant retrieval; backend ACL filter şart.
Vector DB RAGde ne yapar?	Relevant external context retrieval sağlar.

54. Uygulama alıştırmaları

- 1000 sentence embedding üretip brute-force cosine search yaz.
- FAISS IndexFlatIP ile aynı aramayı yap ve sonuçları karşılaştır.
- Normalized vs unnormalized dot-product ranking farkını gözlemle.
- Chunk size 200/500/1000 için retrieval recall karşılaştır.
- Metadata filter ile language/tenant scoped retrieval kur.
- Dense-only ve BM25-only retrieval sonuçlarını kıyasla.
- RRF ile hybrid search yap.
- Top-30 retrieve + top-5 rerank pipelineı tasarla.
- Golden query set ile recall@5, recall@20 ve MRR ölç.
- Embedding model migration için blue/green index planı yaz.

55. Mini portföy projesi

Proje: Production-Grade Hybrid Vector Retrieval for RAG

Dosya	İçerik
src/ingest.py	Parse, chunk, stable IDs
src/embed.py	Batch embeddings + version tag
src/index.py	FAISS/Chroma index abstraction
src/retrieve.py	ANN + metadata filter
src/hybrid.py	BM25 + vector fusion
src/rerank.py	Cross-encoder reranking

src/evaluate.py	Recall@k, MRR, latency
data/golden.jsonl	Query-relevant chunk labels
tests/	ACL, delete, versioning tests
README.md	Architecture + benchmark results

CV cümlesi
Built a hybrid vector retrieval layer for RAG with versioned embeddings, ANN indexing, metadata access filters, BM25 fusion, reranking and golden-set retrieval evaluation.

56. Final cheat sheet

Kavram	Hatırlanacak fikir
Embedding	Semantic dense vector
Cosine	Angle similarity
Dot product	Vector alignment + magnitude
L2	Euclidean distance
ANN	Approximate fast search
HNSW	Graph-based ANN
IVF	Coarse cluster lists
PQ	Vector compression
Metadata filter	Scope/ACL
Hybrid	Dense + lexical
Reranker	High-precision second stage
MMR	Relevance + diversity
Recall@k	Retriever coverage
FAISS	Vector index library
Vector DB	Index + metadata + lifecycle
RAG	Retrieve external context for generation

Bu modülden sonra
Recommendation Systems bölümünde embedding ve nearest-neighbor retrieval bilgisini user-item candidate generation ve similar-item sistemlerine taşıyacaksın.